

# Una Breve Introducci3n a los Algoritmos

Mem3ticos Pablo Moscato Carlos Cotta

## 1. Introducci3n y notas hist3ricas

La denominaci3n gen3rica de ‘Algoritmos Mem3ticos’ (AMs) se utiliza para abarcar una amplia clase de metaheur3sticas (es decir, m3todos de prop3sito general destinados a guiar una heur3stica subyacente). El m3todo se basa en una poblaci3n de agentes y ha demostrado ser un 3xito pr3ctico en una variedad de dominios de problemas y en particular para la soluci3n aproximada de problemas de Optimizaci3n NP.

A diferencia de los m3todos tradicionales de Computaci3n Evolutiva (CE), los AMs se preocupan intr3nsecamente por explotar todo el conocimiento disponible sobre el problema en estudio. La incorporaci3n del conocimiento del dominio del problema no es un mecanismo opcional, sino una caracter3stica fundamental que caracteriza a los AMs. Esta filosof3a de funcionamiento queda perfectamente ilustrada por el t3rmino “mem3tico”. Acu3nada por R. Dawkins [52], la palabra ‘meme’ denota un an3logo al gen en el contexto de la evoluci3n cultural [154]. En palabras de Dawkins:

“Ejemplos de memes son melod3as, ideas, frases pegadizas, modas de ropa, formas de hacer vasijas o de construir arcos. Al igual que los genes se propagan en el acervo gen3tico saltando de cuerpo en cuerpo a trav3s de espermatozoides u3 ovulos, los memes se propagan en el acervo de memes saltando de cerebro en cerebro a trav3s de un proceso que, en un sentido amplio, puede llamarse imitaci3n.”

Esta caracterizaci3n de un meme sugiere que en los procesos de evoluci3n cultural, la informaci3n no se transmite simplemente inalterada entre los individuos. Por el contrario, es procesada y mejorada por las partes comunicantes. Esta mejora se logra en los AMs mediante la incorporaci3n de heur3sticas, algoritmos de aproximaci3n, t3cnicas de b3squeda local, operadores de recombinaci3n especializados, m3todos exactos truncados, etc. En esencia, la mayor3a de los AMs pueden interpretarse como una estrategia de b3squeda en la que una poblaci3n de agentes optimizadores cooperan y compiten [169]. El 3xito de los AMs probablemente pueda explicarse como una consecuencia directa de la sinergia de los diferentes enfoques de b3squeda que incorporan.

La característica más crucial y distintiva de los AMs, la inclusión del conocimiento del problema mencionada anteriormente, también está respaldada por sólidos resultados teóricos. Como Hart y Belew [88] afirmaron inicialmente y Wolpert y Macready [224] popularizaron más tarde en el llamado *Teorema de No Hay Almuerzo Gratis* (No-Free-Lunch Theorem), un algoritmo de búsqueda se desempeña estrictamente de acuerdo con la cantidad y calidad del conocimiento del problema que incorpora. Este hecho sustenta claramente la explotación del conocimiento del problema intrínseca a los AMs. Dado que el término hibridación [42] se utiliza comúnmente para denotar el proceso de incorporación de conocimiento del problema, no es sorprendente que los AMs a veces también se llamen ‘Algoritmos Evolutivos Híbridos’ [51]. Uno de los primeros algoritmos a los que se asignó la etiqueta de AM data de 1988 [169], y fue considerado por muchos como un híbrido de Algoritmos Genéticos (AGs) tradicionales y Recocido Simulado (Simulated Annealing). Parte de la motivación inicial fue encontrar una salida a las limitaciones de ambas técnicas en un problema de optimización combinatoria bien estudiado: el problema del Agente Viajero Euclidiano Mínimo (*Min ETSP*). Según los autores, la inspiración original provino de los torneos de juegos de computadora [96] utilizados para estudiar “la evolución de la cooperación” [6] (véase también [171] y [164] para resultados teóricos más recientes en este campo). Ese enfoque tenía varias características que anticiparon muchos algoritmos actuales en la práctica de hoy. La fase competitiva del algoritmo se basó en la nueva asignación de puntos de búsqueda en el espacio de configuraciones, un proceso que involucraba una “batalla” por la supervivencia seguida de la llamada “clonación”, que tiene una gran similitud con los algoritmos más recientes de ‘ir con los ganadores’ [4] [182]. La fase cooperativa seguida de búsqueda local podría llamarse mejor “ir con los ganadores locales” ya que los agentes optimizadores estaban dispuestos con una topología de una red toroidal bidimensional. Después de los experimentos informáticos iniciales, se derivó una idea sobre la relevancia particular que la organización “espacial”, cuando se acoplaba con un conjunto apropiado de reglas, tenía para el rendimiento general de los procesos de búsqueda poblacional. Unos meses más tarde, Moscato y Norman describieron que compartían puntos de vista similares con otros investigadores [157] [74] y otros autores que proponían “modelos de islas” para los AGs. La especialización ahora se reconoce como el “catalizador” responsable de una variedad de fenómenos [164] [163]. Este es un tema de investigación importante, actualmente solo comprendido de una manera bastante heurística. Sin embargo, se han obtenido algunos resultados propios de indecidibilidad para problemas relacionados [80] que dan alguna esperanza a un tratamiento más formal.

Menos de un año después, en 1989, Moscato y Norman identificaron a varios autores que también fueron pioneros en la introducción de heurísticas para mejorar las soluciones antes de recombinarlas [158] [73] (ver otras

## 2

referencias y la discusión en [154]). Particularmente provenientes del campo de los AGs, varios autores estaban introduciendo conocimiento del dominio del problema en una variedad de formas. En [154] se introdujo por primera vez la denominación de ‘algoritmos meméticos’. También se sugirió que la evolución cultural podría ser una mejor metáfora de trabajo para estas metaheurísticas para evitar el pensamiento “biológicamente restringido” que limitaba el progreso en ese momento.

Diez años después, aunque lamentablemente bajo diferentes nombres, los AMs se han convertido en un importante enfoque de optimización, con varios éxitos en una variedad de problemas clásicos de optimización NP-duros. Nuestro objetivo es proporcionar una introducción actualizada y autónoma a los AMs, centrándonos en sus entresijos técnicos y características formales, pero sin perder la perspectiva de su aplicación práctica y problemas de investigación abiertos.

## 2. Algoritmos Meméticos

Antes de proceder a la descripción de los AMs, es necesario proporcionar algunos conceptos y definiciones básicos. Varias nociones introducidas en la primera subsección están fuertemente relacionadas con el campo de la complejidad computacional. Sin embargo, pueden ser presentadas de una manera y a un ritmo ligeramente diferentes en aras del desarrollo posterior. Estos conceptos básicos darán lugar a las nociones de búsqueda local y búsqueda basada en poblaciones, sobre las cuales se fundan los AMs. Esta última clase de búsqueda establece el escenario para la recombinación, un mecanismo crucial en el funcionamiento de los AMs que será estudiado con cierta profundidad. Finalmente, se presentarán algunas pautas para el diseño de AMs.

### 2.1. Conceptos Básicos

Un algoritmo es un procedimiento detallado paso a paso para resolver un problema computacional. Un problema computacional  $P$  denota una clase de tareas realizables algorítmicamente, y tiene un conjunto de dominio de entrada de instancias denotado como  $I_P$ . Para cada instancia  $x \in I_P$ , hay un conjunto asociado  $sol_P(x)$  que denota las soluciones factibles para el problema  $P$  dada la instancia  $x$ . El conjunto  $sol_P(x)$

también se conoce como el conjunto de soluciones aceptables o válidas.

Se espera que entreguemos un algoritmo que resuelva el problema  $P$ ; esto significa que nuestro algoritmo, dada la instancia  $x \in I_P$ , debe devolver al menos un elemento  $y$  de un conjunto de respuestas  $ans_P(x)$  (también llamadas soluciones dadas) que satisfaga los requisitos del problema. Este es el primer problema de diseño a enfrentar. Para ser precisos, dependiendo

### 3

del tipo de respuestas esperadas, los problemas computacionales se pueden clasificar en diferentes categorías; por ejemplo:

encontrar *todas* las soluciones en  $sol_P(x)$ , es decir, problemas de enumeración.

contar cuántas soluciones existen en  $sol_P(x)$ , es decir, problemas de conteo.

determinar si el conjunto  $sol_P(x)$  está vacío o no, es decir, problemas de decisión.

encontrar una solución en  $sol_P(x)$  maximizando o minimizando una función dada, es decir, problemas de optimización.

En este capítulo, nos centraremos en la última posibilidad, es decir, un problema se considerará resuelto al encontrar una cierta solución factible, es decir, ya sea encontrando un óptimo  $y^* \in sol_P(x)$  o dando una indicación de que tal solución factible no existe. Por lo tanto, es conveniente en muchas situaciones definir una función de factibilidad booleana  $feasible_P(x, y)$  con el fin de identificar si una solución dada  $y \in ans_P(x)$  es aceptable para una instancia  $x \in I_P$  de un problema computacional  $P$ , es decir, comprobando si  $y \in sol_P(x)$ .

Se dice que un algoritmo resuelve el problema  $P$  si puede cumplir esta condición para cualquier instancia dada  $x \in I_P$ . Esta definición es ciertamente demasiado amplia, por lo que es necesaria una caracterización más restrictiva para nuestros problemas de interés. Esta caracterización se proporciona al restringirnos a los llamados *problemas de optimización combinatoria*. Estos constituyen una subclase especial de problemas computacionales en los que para cada instancia  $x \in I_P$ :

la cardinalidad de  $sol_P(x)$  es finita.

cada solución  $y \in sol_P(x)$  tiene un valor entero de bondad  $m_P(y, x)$ ,

obtenido por medio de una función objetivo asociada  $m_P$ .

un orden parcial  $\leq_P$  se define sobre el conjunto de valores de bondad devueltos por la función objetivo, permitiendo determinar cuál de dos valores de bondad es preferible.

Una instancia  $x \in I_P$  de un problema de optimización combinatoria  $P$  se resuelve encontrando la mejor solución  $y^* \in \text{sol}_P(x)$ , es decir, encontrando una solución  $y^*$  tal que no exista otra solución  $y \leq_P y^*$  si  $\text{sol}_P(x)$  no está vacío. Es muy común que  $\leq_P$  defina un orden total. En este caso, la mejor solución es la que maximiza (o minimiza) la función objetivo.

Como ejemplo de un problema de optimización combinatoria, considere el Problema de la Mochila Múltiple 0-1 (0-1 MKP). Cada instancia  $x$  de

4

este problema está definida por un vector de beneficios  $V = \{v_0, \dots, v_{n-1}\}$ , un vector de capacidades  $C = \{c_0, \dots, c_{m-1}\}$ , y una matriz de restricciones de capacidad  $M = \{m_{ij} : 0 \leq i < m, 0 \leq j < n\}$ . Intuitivamente, el problema consiste en seleccionar un conjunto de objetos para maximizar el beneficio de este conjunto sin violar las restricciones de capacidad. Si los objetos se indexan con los elementos del conjunto  $N_n = \{0, 1, \dots, n-1\}$ , el conjunto de respuestas  $\text{ans}_P(x)$  para una instancia  $x$  es simplemente el conjunto potencia de  $N_n$ , es decir, cada subconjunto de  $N_n$  es una respuesta posible. Además, el conjunto de respuestas factibles  $\text{sol}_P(x)$  está compuesto por aquellos subconjuntos cuyo vector de incidencia  $B$  verifica  $M \cdot B \leq C^0$ .

Finalmente, la función objetivo se define como  $m_P(y, x) = \sum_{i \in y} v_i$ , es decir, la suma de los beneficios para todos los objetos seleccionados, siendo el objetivo maximizar este valor.

Nótese que, asociado a un problema de optimización combinatoria, podemos definir su versión de decisión. Para formular el problema de decisión, se considera un valor entero de bondad  $K$ , y en lugar de intentar encontrar la mejor solución de la instancia  $x$ , preguntamos si  $x$  tiene una solución cuya bondad es igual o mejor que  $K$ . En el ejemplo anterior, podríamos preguntar si existe una solución factible  $y$  tal que su beneficio asociado sea igual o mejor que  $K$ .

## 2.2. Espacios de Búsqueda

Como se mencionó anteriormente, habiendo definido el concepto de problema de optimización combinatoria, el objetivo es encontrar al menos una de las soluciones óptimas para una instancia dada. Para este

proposito, se debe utilizar un algoritmo de búsqueda. Antes de discutir los algoritmos de búsqueda, deben discutirse tres entidades. Estas son el *espacio de búsqueda*, la *relacion de vecindad* y la *funcion guia*. Es importante considerar que, para cualquier problema computacional dado, estas tres entidades pueden ser instanciadas de varias maneras, dando lugar a diferentes tareas de optimización.

para resolver un problema de optimización necesitamos un algoritmo de búsqueda

hay que definir 3 conceptos

Comencemos definiendo el concepto de espacio de búsqueda para un problema combinatorio  $P$ . Para hacerlo, consideramos un conjunto  $S_P(x)$ , cuyos elementos tienen las siguientes propiedades:

Cada elemento  $s \in S_P(x)$  representa al menos una respuesta en  $ans_P(x)$ .

Para problemas de decisión: al menos un elemento de  $sol_P(x)$  que re presenta una respuesta de 'S' debe estar representado por un elemento en  $S_P(x)$ .

Para problemas de optimización: al menos un elemento 'óptimo'  $y^*$  de  $sol_P(x)$  está representado por un elemento en  $S_P(x)$ .

espacio de búsqueda: un conjunto  $S_P(x)$

1: cada elemento (s) tiene que representar al menos una respuesta posible

2: en problemas de decision deve haver al menos un si

3: en problemas de optimización deve haver al menos una soluicon posible

Cada elemento de  $S_P(x)$  se denominará *configuración*, estando relacionado con una respuesta en  $ans_P(x)$  por una función de crecimiento  $g : S_P(x) \rightarrow ans_P(x)$ . Note que el primer requisito se refiere a  $ans_P(x)$  y no a  $sol_P(x)$ , es decir, algunas configuraciones en el espacio de búsqueda pueden corresponder a soluciones no factibles. Por lo tanto, el algoritmo de búsqueda puede necesitar estar preparado para lidiar con este hecho. Si se han logrado estos requisitos, decimos que tenemos una *representación válida* o *formulación válida* del problema. Para

simplificar, simplemente escribiremos  $S$  para referirnos a  $S_P(x)$  cuando  $x$  y  $P$  estén claros por el contexto. A las personas que utilizan metáforas de inspiración biológica les gusta llamar a  $S_P(x)$  el *espacio de genotipos* y  $ans_P(x)$  denota el *espacio de fenotipos*, por lo que nos referimos apropiadamente a  $g$  como la *función de crecimiento*.

configuración: elementos del espacio de búsqueda  
 puede ser no factible (inválida)  
 - espacio de búsqueda: genotipo  
 -  $ans_P(x)$  fenotipo  
 $ans_P(x)$ : respuestas posibles  
 $sol_P(x)$ : soluciones factibles o válidas

Para ilustrar esta noción de espacio de búsqueda, considere nuevamente el caso del 0-1 MKP. Dado que las soluciones en  $ans_P(x)$  son subconjuntos de  $N_n$ , podemos definir el espacio de búsqueda como el conjunto de vectores binarios de  $n$  dimensiones. Cada vector representaría el vector de incidencia de un cierto subconjunto, es decir, la función de crecimiento  $g$  se define como  $g(s) = g(b_0 b_1 \dots b_{n-1}) = \{i | b_i = 1\}$ . Como se mencionó anteriormente, muchos vectores binarios pueden corresponder a conjuntos de objetos infactibles. Otra posibilidad es definir el espacio de búsqueda como el conjunto de permutaciones de elementos en  $N_n$  [77]. En este caso, la función de crecimiento puede consistir en aplicar un algoritmo de construcción voraz [45], considerando los objetos en el orden proporcionado por la permutación. A diferencia del espacio de búsqueda binario mencionado anteriormente, todas las configuraciones representan soluciones factibles en este caso.

El papel del espacio de búsqueda es proporcionar un “terreno” sobre el cual actuará el algoritmo de búsqueda y, por supuesto, moverse indirectamente en el conjunto imagen  $ans_P(x)$ . Importantes propiedades del espacio de búsqueda que afectan la dinámica del algoritmo de búsqueda están relacionadas con las relaciones de accesibilidad entre las configuraciones. Estas relaciones dependen de una función de vecindad  $N : S \rightarrow 2^S$ . Esta función asigna a cada elemento  $s \in S$  un conjunto  $N(s) \subseteq S$  de configuraciones vecinas de  $s$ . El conjunto  $N(s)$  se llama *vecindad* de  $s$  y cada miembro  $s' \in N(s)$  se llama un *vecino* de  $s$ .

espacio de búsqueda es como una mapa o terreno para moverse por

el,

funcion de vecindad  $N$  : configuraciones a las que se puede llegar con un pequeño cambio

Debe notarse que la vecindad depende de la instancia, por lo que la notación  $N(s)$  es una forma simplificada de  $N_P(s, x)$  ya que queda claro por el contexto. Los elementos de  $N(s)$  no necesitan estar enumerados explícitamente. De hecho, es muy habitual definirlos implícitamente refiriéndose a un conjunto de *movimientos posibles*, que definen transiciones entre configuraciones. Los movimientos se definen generalmente como modificaciones “locales” de alguna parte de  $s$ , donde la “localidad” se refiere al hecho de que el movimiento se hace sobre una sola solución para obtener otra solución única. Esta “localidad”, es uno de los ingredientes clave de la *busqueda local*, y de hecho tambien ha dado el nombre a todo el paradigma de busqueda.

movimientos: canbiso locales pequeños para obtener otra.  
localidad

Como ejemplos de definiciones concretas de vecindad, considere las dos representaciones de soluciones para el 0-1 MKP presentadas arriba. En el primer caso (representación binaria), los movimientos se pueden definir como cambiar los valores de un número de bits. Si sólo se modifica un bit a la vez, la estructura de vecindad resultante es el hipercubo binario  $n$ -dimensional. En el segundo caso (representación de permutación), los movimientos se pueden definir como el intercambio de dos posiciones en la permutación. Por lo tanto, dos configuraciones son vecinas si, y sólo si, difieren en exactamente dos posiciones.

movimiento: cambiar un solo bit  
movimiento: intercambiar posiciones

Esta definición de localidad presentada anteriormente no está necesariamente relacionada con la “cercanía” bajo algún tipo de relación de distancia entre configuraciones (excepto en la situación tautológica en la que la distancia entre dos configuraciones  $s$  y  $s'$  se define como el número de movimientos necesarios para llegar a  $s'$  desde  $s$ ). Como cuestión de hecho, es posible dar ejemplos comunes de definiciones de vecindad muy complejas no relacionadas con medidas de distancia intuitivas.

2 configuraciones sean vecinas no es parecidas o cercanas



Una característica importante que debe considerarse al seleccionar la clase de movimientos a usar en el algoritmo de búsqueda es su “*ergodicidad*”, es decir, la capacidad, dado cualquier  $s \in S$  de encontrar una secuencia de movimientos que pueda alcanzar cualquier otra configuración  $s' \in S$ . En muchas situaciones, esta propiedad es evidente por sí misma y no se requiere una demostración explícita. Es importante ya que incluso si se tiene una representación válida (recuerde la definición anterior), es necesario garantizar que a priori al menos una solución óptima es alcanzable desde cualquier solución inicial dada. Nuevamente, considere la representación binaria de soluciones para una instancia 0-1 MKP. Si los movimientos se definen como giros de un solo bit, se ve fácilmente que cualquier configuración  $s'$  puede alcanzarse desde otra configuración  $s$  en exactamente  $h$  movimientos, donde  $h$  es la distancia de Hamming entre estas configuraciones. Sin embargo, este no es siempre el caso.

ergodicidad: partiendo desde cualquier configuración inicial se debe poder llegar a otra configuración mediante una secuencia de elementos

La última entidad que debe definirse es la *función guía*. Para hacerlo, requerimos un conjunto  $F$  cuyos elementos se denominan *valores de aptitud* (típicamente  $F \subseteq \mathbb{R}$ ), y un orden parcial  $\leq_F$  en  $F$  (típicamente, pero no siempre,  $\leq_F$  coincide con  $\leq$ ). La función guía se define como una función  $F_g : S \rightarrow F$  que asocia a cada configuración  $s \in S$  un valor  $F_g(s)$  que evalúa la calidad de la solución. El comportamiento del algoritmo de búsqueda estará “controlado” por estos valores de aptitud.

funcion guia( $F_g$ ): asigna fitness a cada configuración ,  
mide la calidad de cada configuración ,  
el algoritmo usará estos valores para decidir a dónde moverse

Nótese que para los problemas de optimización hay una obvia conexión directa entre la función guía  $F_g$  y la función objetivo  $m_P$ . De hecho, es muy común forzar esta relación hasta el punto de que ambos términos se consideran usualmente equivalentes. Sin embargo, esta equivalencia no es necesaria y, en muchas situaciones, ni siquiera deseable. Para problemas de decisión, ya que una solución es una respuesta de ‘Sí’ o ‘No’, las funciones guía asociadas usualmente toman la forma de *distancia a la satisfactibilidad*.

Un ejemplo típico es el *Problema de Satisfacibilidad Booleana*, es decir, determinar si una expresión booleana en forma normal conjuntiva es satisfacible. En este caso, las soluciones son asignaciones de valores booleanos a variables, y la función objetivo  $m_P$  es una función binaria que devuelve 1 si la solución satisface la expresión booleana, y devuelve 0 en caso contrario. Esta función objetivo podría ser utilizada como función guía. Sin embargo, una elección mucho más típica es usar el número de cláusulas satisfechas en la configuración actual como función guía, es decir,  $F_g(s) = \sum_i f_i(s)$ , la suma sobre los índices de cláusulas  $i$  de  $f_i(s)$ , definida como  $f_i(s) = 0$  para una cláusula  $i$  aún no satisfecha, y  $f_i(s) = 1$  si la cláusula  $i$  está satisfecha. Por lo tanto, el objetivo es maximizar este número. Nótese que la función guía es en este caso la función objetivo del problema de Optimización NP asociado llamado *Max SAT*.

funcion objetivo: juez final (problema de decision si o no)

funcion guia: cuan cerca

La diferenciación anterior entre la función objetivo y la función guía también es muy importante en el contexto de los problemas de optimización con restricciones, es decir, problemas para los cuales, en general,  $sol_P(x)$  se elige como un subconjunto propio de  $ans_P(x)$ . Dado que la función de crecimiento establece un mapeo de  $S$  a  $ans_P(x)$ , el algoritmo de búsqueda podría necesitar procesar tanto soluciones factibles (cuyos valores de bondad están bien definidos) como soluciones infactibles (cuyos valores de bondad están mal definidos en general). En muchas implementaciones de AMs para estos problemas, una función guía se define como una suma ponderada del valor de la función objetivo y la distancia a la factibilidad (que da cuenta de las restricciones). Típicamente, se asigna un mayor peso a las restricciones, a fin de dar preferencia a la factibilidad sobre la optimalidad. Abundan varias otras soluciones a este problema, incluyendo el recurso a técnicas multiobjetivo.

problemas con restricción(donde muchas soluciones son invalidas)

funcion guia: suma ponderada + penalización

el algoritmo priorizará encontrar soluciones factibles que optimas pero invalidas

La combinación de una cierta instancia de problema y las tres entidades definidas arriba induce un llamado *panorama de aptitud* o paisaje de

búsqueda [106]. Esencialmente, un panorama de aptitud se puede definir como un digrafo ponderado, en el que los vértices son configuraciones del espacio de búsqueda  $S$ , y los arcos conectan configuraciones vecinas. Los pesos son la diferencia de la función guía de las configuraciones extremas. La búsqueda puede verse así como el proceso de “navegar” por el panorama de aptitud utilizando la información proporcionada por la función guía. Esta es una metáfora muy poderosa; permite interpretar en términos de objetos topográficos bien conocidos como *picos*, *valles*, *mesetas*, etc., de gran utilidad para visualizar el progreso de la búsqueda y captar los factores que afectan el rendimiento del proceso. En particular, la importante noción de *óptimo local* está asociada a esta definición de panorama de aptitud. Para ser precisos, un *óptimo local* es un vértice del panorama de aptitud cuyo valor de la función guía es mejor que los valores de todos sus vecinos. Nótese que diferentes movimientos definen diferentes vecindades y, por lo tanto, diferentes panoramas de aptitud, incluso cuando se considera la misma instancia del problema. Por esta razón, la noción de *óptimo local* no es intrínseca a una instancia del problema como, a veces, se considera erróneamente.

panorama de aptitud(fitness landscape)  
 mapa con picos , óptimo local

## 2.3. Búsqueda Local vs. Basada en Poblaciones

Las definiciones presentadas en la subsección anterior conducen naturalmente a la noción de *algoritmo de búsqueda local*. Un algoritmo de búsqueda local comienza desde una configuración  $s_0 \in S$ , generada al azar o construida por algún otro algoritmo. Posteriormente, itera utilizando en cada paso una transición basada en la vecindad de la configuración actual. Las transiciones que conducen a configuraciones preferibles (según el orden parcial  $\leq_F$ ) son aceptadas, es decir, la configuración recién generada pasa a ser la configuración actual en el siguiente paso. De lo contrario, se mantiene la configuración actual. Este proceso se repite hasta que se cumple un determinado criterio de terminación. Los criterios típicos son la realización de un número preespecificado de iteraciones, no haber encontrado ninguna mejora en las últimas  $m$  iteraciones, o incluso mecanismos más complejos basados en la estimación de la probabilidad de estar en un *óptimo local* [44].

busqueda local:

1 comienza con una solucion inicial y paso a paso mira a sus vesinos

2 si encuentra un vecino mejor se mueve a el , si no se queda donde esta

3 si no se queda donde esta

criterio de parada: alcanzar maximo de iteraciones , estanacado sin mejorar , calcular que ya esta el ultimo

Debido a estas característias, el enfoque se llama metafóricamente “*es calada de colinas*”. Todo el proceso se esboza en la Fig. 1.

Procedure Local-Search-Engine (current)

begin

repeat

new := GenerateNeighbor(current);

if (F\_g(new) <= F\_g(current)) then

current := new;

endif

until TerminationCriterion();

return current;

end

func busqueda\_local():

Figura 1: Un Algoritmo de Búsqueda Local

La selección del tipo particular de movimientos (también conocidos como mutación en el contexto de los AGs) a utilizar depende ciertamente de las características específicas del problema y de la representación elegida. No hay un consejo general para esto, ya que es una cuestión del tiempo de computadora disponible para todo el proceso, así como de otras decisiones algorítmicas que incluyen la facilidad de codificación, etc. En algunos casos algunos movimientos son notorios, por ejemplo, puede ser el cambio del valor de una sola variable o el intercambio de los valores de dos variables diferentes. A veces el “paso” también puede estar compuesto por una cadena de transiciones. Por ejemplo, en relación con los AMs, Radcliffe y Surry

introdujeron el concepto de *Mutación Mínima Binomial*, donde el número de mutaciones a realizar se selecciona de acuerdo con cierta distribución binomial [189]. En el contexto de los panoramas de aptitud, esto equivale a una redefinición de la relación de vecindad, considerando dos configuraciones como vecinas cuando existe una cadena de transiciones conectándolas.

movimientos (mutacion en ag) - no hay una regla para elegirlos

depende de:

tiempo de computacion ,

problema

lo facil que sea programar

ejm:

cambiar un bit ,

cadena de varios cambio ejm mutacion minima binomial

Los algoritmos de búsqueda local se caracterizan por lo tanto por mantener una sola configuración a la vez. La generalización inmediata de este comportamiento es el mantenimiento simultáneo de  $k$ , ( $k \geq 2$ ) configuraciones. El término *algoritmos de búsqueda basados en poblaciones* ha sido acuñado para denotar técnicas de búsqueda que se comportan de esta manera.

búsqueda local : una sola solución a la vez

búsqueda basado en poblaciones: varias soluciones a la vez

La disponibilidad de varias configuraciones a la vez permite el uso de nuevos y potentes mecanismos para atravesar el panorama de aptitud además del operador de mutación estándar. El más popular de estos

mecanismos, el operador de recombinación, será estudiado con más profundidad en la siguiente sección.

ventaja: no solo usamos la mutación, sino que podemos usar la reconstrucción

En cualquier caso, nótese que el funcionamiento general de las técnicas de búsqueda basadas en poblaciones es muy similar al pseudocódigo representado en la Fig. 1. Como cuestión de hecho, un algoritmo basado en poblaciones puede imaginarse como un procedimiento en el que visitamos secuencialmente vértices de un hipergrafo. Cada vértice del hipergrafo representa un conjunto de configuraciones en  $S_P(x)$ , es decir, una población. El siguiente vértice a ser visitado, es decir, la nueva población, puede elegirse de acuerdo con la composición de las vecindades de los diferentes mecanismos de transición utilizados en el algoritmo de población. A pesar de la analogía con la búsqueda local, está ampliamente aceptado en la literatura científica aplicar la denominación 'local' sólo a los algoritmos de búsqueda de una configuración a la vez. Por esta razón, el término 'local' se utilizará con esta interpretación en el resto del artículo.

local: se usará estrictamente para algoritmos que a una sola solución a la vez

búsqueda en poblaciones: búsqueda local pero en vez de nodos (soluciones individuales) en un grafo, visita hiper-nodo (poblaciones enteras) de un hipergrafo

## 2.4. Recombinación

Como se mencionó en la sección anterior, la búsqueda local se basa en la aplicación de un operador de mutación a una sola configuración. A pesar de la aparente simplicidad de este mecanismo, la búsqueda local "basada en mutación" ha revelado ser un mecanismo muy poderoso para obtener soluciones de buena calidad para problemas NP-duros (por ejemplo, véase [62] [199]). Por esta razón, algunos investigadores han intentado proporcionar una base teóricamente más sólida a esta clase de búsqueda. En esta línea, vale la pena mencionar la definición de la clase de Búsqueda Local Polinomial (PLS) de Johnson et al. [104]. Básicamente, esta clase de complejidad comprende un problema y un paisaje de búsqueda asociado de tal manera que podemos decidir en tiempo polinomial si podemos encontrar una solución mejor en la vecindad. Desafortunadamente, es muy probable que ningún

problema NP-duro est e contenido en la clase PLS, ya que eso implicar a que  $NP = co-NP$  [226], una conjetura que generalmente se asume falsa. Este hecho ha justificado la b usqueda de mecanismos de b usqueda adicionales para ser utilizados como operadores independientes o como complementos de la mutaci on est andar.

En esta l nea, recordemos que la b usqueda basada en poblaciones permiti o la definici on de operadores de movimiento generalizados denominados operadores de recombinaci on. En esencia, la recombinaci on se puede definir como un proceso en el que un conjunto  $S_{par}$  de  $n$  configuraciones (informalmente referidas como “padres”) es manipulado para crear un conjunto  $S_{desc} \subseteq sol_P(x)$  de  $m$  nuevas configuraciones (informalmente denominadas “descendientes”). La creaci on de estos descendientes implica la identificaci on y combinaci on de caracter isticas extra idas de los padres.

Llegados a este punto, es posible considerar las propiedades de inter es que pueden ser exhibidas por los operadores de recombinaci on [189]. La primera propiedad, el *respeto*, representa el lado de explotaci on de la recombinaci on. Se dice que un operador de recombinaci on es *respetuoso*, con respecto a un tipo particular de caracter isticas de las configuraciones, si, y s olo si, genera descendientes que llevan todas las caracter isticas b asicas comunes a todos los padres. N otese que, si todas las configuraciones parentales son id enticas, un operador de recombinaci on respetuoso est a obligado a devolver la misma configuraci on como descendiente. Esta propiedad se denomina *pu reza*, y se puede lograr incluso cuando el operador de recombinaci on no es generalmente respetuoso.

Por otro lado, la *mezcla* (assortment) representa el lado exploratorio de la recombinaci on. Se dice que un operador de recombinaci on es *propia mente mezclador* si, y s olo si, puede generar descendientes que portan cualquier combinaci on de caracter isticas compatibles tomadas de los padres. Se dice que la mezcla es *d ebil* si es necesario realizar varias recombinaciones dentro de la descendencia para lograr este efecto.

Finalmente, la *transmisi on* es una propiedad muy importante que captura el papel intuitivo de la recombinaci on. Se dice que un operador *transmite* si cada caracter istica exhibida por la descendencia est a presente en al menos uno de los padres. As i, un operador de recombinaci on transmisor combina la informaci on presente en los padres pero no introduce nueva informaci on. Esta  ultima tarea se deja normalmente al operador de mutaci on. Por esta raz on, se dice que un operador de recombinaci on no transmisor *introduce una mutaci on impl cita*.

Las tres propiedades anteriores son suficientes para describir el

comportamiento abstracto de entrada/salida de un operador de recombinación con respecto a algunas características particulares. Proporciona una caracterización de los posibles descendientes que puede producir el operador. Sin embargo, existen otros aspectos del funcionamiento de la recombinación que deben ser estudiados. En particular, es interesante considerar cómo se abor

da la construcción de  $S_{desc}$ .

En primer lugar, se dice que un operador de recombinación es *ciego* si no tiene otra entrada que  $S_{par}$ , es decir, no utiliza ninguna información de la instancia del problema. Esta definición es ciertamente muy restrictiva, y por lo tanto a veces se relaja para permitir que el operador de recombinación utilice información sobre las restricciones del problema (con el fin de construir descendientes factibles), y posiblemente los valores de aptitud de las configuraciones  $y \in S_{par}$  (con el fin de sesgar la generación de descendientes hacia los mejores padres). Un ejemplo típico de un operador de recombinación ciego es el cruce Uniforme clásico [209]. Este operador está definido en los espacios de búsqueda  $S = \Sigma^n$ , es decir, cadenas de  $n$  símbolos tomados de un alfabeto  $\Sigma$ . La construcción del descendiente se realiza seleccionando al azar en cada posición uno de los símbolos que aparecen en esa posición en cualquiera de los padres. Esta selección aleatoria puede ser totalmente uniforme o puede estar sesgada de acuerdo con los valores de aptitud de los padres, como se mencionó anteriormente. Además, la selección puede hacerse con el fin de forzar la factibilidad (por ejemplo, considerar la representación binaria de soluciones en el 0-1 MKP). Nótese que, en este caso, el operador resultante no es ni respetuoso ni transmisor en general.

El uso de operadores de recombinación ciegos ha sido justificado generalmente sobre la base de no introducir un sesgo excesivo en el algoritmo de búsqueda, previniendo así una convergencia extremadamente rápida a soluciones subóptimas. Sin embargo, esto es cuestionable. Primero, note que el comportamiento del algoritmo está de hecho sesgado por la elección de la representación y la mecánica de los operadores particulares. Segundo, existen mecanismos ampliamente conocidos (por ejemplo, el aislamiento es parcial) para obstaculizar estos problemas. Finalmente, puede ser mejor obtener rápidamente una solución subóptima y reiniciar el algoritmo que usar operadores ciegos durante mucho tiempo en busca de un comportamiento asintóticamente óptimo (que ni siquiera está garantizado en la mayoría de los casos).

Los operadores de recombinación que utilizan conocimiento del problema se denominan comúnmente *heurísticos* o *híbridos*. En estos operadores, la información del problema se utiliza para guiar el proceso de construcción de los descendientes. Esto se puede hacer de una



plétora de maneras para cada problema, por lo que es difícil proporcionar una taxonomía de los operadores de recombinación heurísticos. Sin embargo, existen dos aspectos principales en los que se puede inyectar conocimiento del problema: la selección de las características parentales que se transmitirán al descendiente, y la selección de las características no parentales que se le añadirán. Un operador de recombinación heurística puede centrarse en uno de estos aspectos, o en ambos simultáneamente.

Como ejemplo de operador de recombinación heurística centrado en el primer aspecto, se debe mencionar la Recombinación Dinásticamente Óptima (DOR) [43]. Este operador explora el *conjunto de potencial dinástico* (es decir, los posibles hijos) de las configuraciones que se están recombinando, a fin de encontrar el mejor miembro de este conjunto (nótese que, dado que las configuraciones en el potencial dinástico están compuestas enteramente de características tomadas de cualquiera de los padres, este es un operador transmisor). Esta exploración se realiza utilizando un algoritmo A\* subordinado para minimizar el coste de atravesar el potencial dinástico. El objetivo de este operador es, por lo tanto, encontrar la mejor combinación de características parentales que den lugar a un hijo factible. Por lo tanto, este operador es *monótono* en el sentido de que cualquier hijo generado es al menos tan bueno como el mejor padre.

Ejemplos de operadores de recombinación heurísticos que se concentran en la selección de características no parentales, se pueden citar los operadores de parcheo-por-compleción-de-forma (patching-by-forma-completion) propuestos por Radcliffe y Surry [188]. Estos operadores se basan en generar un hijo incompleto utilizando un procedimiento no heurístico (por ejemplo, el operador RAR [187]), y luego completar al hijo usando un procedimiento local de escalada de colinas restringido a las características no especificadas (*compleción de forma localmente óptima*) o un procedimiento de búsqueda global que encuentra la mejor solución global que lleva las características especificadas (*compleción de forma globalmente óptima*). Nótese la similitud de este último enfoque con DOR.

Finalmente, existen algunos operadores que intentan explotar el conocimiento en ambos aspectos anteriores. Un ejemplo distinguido es el Cruce de Ensamblaje de Bordes (EAX) [162]. EAX es un operador especializado para el TSP (tanto para instancias simétricas como asimétricas) en el que la construcción del hijo comprende dos fases: la primera implica la generación de un hijo incompleto a través de los llamados *conjuntos E* (subtours con puestos por bordes alternos de cada

padre); posteriormente, estos subtours se fusionan en subtours factibles simples usando un algoritmo de reparación voraz. Los autores de este operador reportaron resultados impresionantes en términos de precisión y velocidad. Tiene algunas similitudes con el operador de recombinación propuesto en [155].

Debe hacerse un comentario final en relación con la complejidad computacional de la recombinación. Está claro que combinar las características de varias soluciones es en general computacionalmente más costoso que modificar una sola solución (es decir, una mutación). Además, la operación de recombinación se invocará usualmente un gran número de veces. Por esta razón, es conveniente (y en muchas situaciones obligatorio) mantenerlo a un bajo coste computacional. Una pauta razonable es considerar un límite superior de  $O(N \log N)$  para su complejidad, donde  $N$  es el tamaño de la entrada (el conjunto  $S_{par}$  y la instancia del problema  $x$ ). Dicho límite es fácilmente asequible para los operadores de recombinación ciegos, siendo el término *cruce* (crossover) un nombre razonable que transmite la baja complejidad

## 13

(aunque no siempre se utiliza en este contexto) de éstos. Sin embargo, este límite puede ser relativamente estricto en el caso de la recombinación heurística, principalmente cuando la epistasis (influencia inter-característica no aditiva sobre el valor de aptitud) está involucrada. Esto admite varias soluciones dependiendo de la heurística particular utilizada. Por ejemplo, DOR tiene un comportamiento de peor caso exponencial, pero se puede hacer asequible eligiendo partes más grandes de información de cada padre (cuanto mayor sea el tamaño de estas partes de información, menor será el número de ellas necesario para completar al hijo) [46]. En cualquier caso, considere que los operadores de recombinación heurísticos proporcionan mejores soluciones que los operadores de recombinación ciegos y, por lo tanto, no necesitan ser invocados el mismo número de veces.

## 2.5. Diseño de un Algoritmo Memético

A la luz de las consideraciones anteriores, es posible proporcionar una plantilla general para un algoritmo memético. Como se mencionó en la Subsección 2.3, esta plantilla es muy similar a la de un procedimiento de búsqueda local que actúa sobre un conjunto de  $|pop| \geq 2$  configuraciones. Esto se muestra en la Fig. 2:

Procedure Population-Based-Search-Engine

begin

    Initialize pop using GenerateInitialPopulation(); repeat

        newpop := GenerateNewPopulation(pop);

        pop := UpdatePopulation(pop, newpop)

```

    if pop has converged then
        pop := RestartPopulation(pop);
    endif
until TerminationCriterion()
end

```

Figura 2: Un Algoritmo de Búsqueda Basado en Poblaciones

Esta plantilla requiere alguna explicación. En primer lugar, el procedimiento `GenerateInitialPopulation` es responsable de crear el conjunto inicial de  $|pop|$  configuraciones. Esto se puede hacer simplemente generando  $|pop|$  configuraciones aleatorias o usando un mecanismo de siembra más sofisticado (por ejemplo, alguna heurística constructiva), por medio del cual se inyectan configuraciones de alta calidad en la población inicial [208] [130]. Otra posibilidad, el `Local-Search-Engine` presentado en la Subsección 2.3 podría utilizarse como se muestra en la Fig. 3:

En cuanto a la función `TerminationCriterion`, se puede definir de ma

14

```

Procedure GenerateInitialPopulation
begin
    Initialize pop using EmptyPopulation();
    parfor j := 1 to popsize do
        i := GenerateRandomConfiguration();
        i := Local-Search-Engine(i);
        InsertInPopulation individual i to pop;
    endparfor
    return pop
end

```

Figura 3: Inyectando soluciones de alta calidad en la población inicial.

nera muy similar al caso de la Búsqueda Local, es decir, estableciendo un límite en el número total de iteraciones, alcanzando un número máximo de iteraciones sin mejora, o habiendo realizado un cierto número de reinicios de población, etc.

El procedimiento `GenerateNewPopulation` es el núcleo del algoritmo memético. Esencialmente, este procedimiento puede verse como un proceso en cadena o tubería (pipeline) que comprende  $n_{op}$  etapas. Cada una de estas etapas consiste en tomar  $arity_{in}^j$  configuraciones de la etapa

anterior, generando  $arity_{out}^j$  configuraciones nuevas aplicando un operador  $op_j$ . Este pipeline está restringido a tener  $arity_{in}^1 = popsize$ . Todo el proceso se esboza en la Fig. 4:

```

Procedure GenerateNewPopulation(pop)
begin
  buffer_0 := pop;
  parfor j := 1 to n_op do
    Initialize buffer_j using EmptyPopulation();
  endparfor
  parfor j := 1 to n_op do
    S_{par}^j := ExtractFromBuffer(buffer_{j-1}, arity_{in}^j); S_{desc}^j :=
    ApplyOperator(op_j, S_{par}^j);
    for z := 1 to arity_{out}^j do
      InsertInPopulation individual S_{desc}^j[z] to buffer_j; endfor
    endparfor;
  return buffer_{n_op}
end

```

Figura 4: El procedimiento GenerateNewPopulation en

tubería. 15

Esta plantilla para el procedimiento GenerateNewPopulation se instancia usualmente en los AGs dejando  $n_{op} = 3$ , usando un operador de selección, recombinación y mutación. Tradicionalmente, la mutación se aplica después de la recombinación, es decir, en cada hijo generado por el operador de recombinación. Sin embargo, si se está utilizando un operador de recombinación heurística, puede ser más conveniente aplicar la mutación antes de la recombinación. Dado que el propósito de la mutación es simplemente introducir nuevas características en el grupo de configuraciones, usarla de antemano también es posible. Además, la combinación inteligente de características realizada por el operador heurístico no se vería perturbada de esta manera.

Esta situación es ligeramente diferente en los AMs. En este caso, es muy común hacer que  $n_{op} = 5$ , insertando un Local-Search-Engine justo después de aplicar  $op_2$  y  $op_4$  (respectivamente recombinación y mutación). Debido a la optimización local realizada después de la mutación, aplicarla después de la recombinación no es tan problemático como en los AGs.

El procedimiento UpdatePopulation se utiliza para reconstruir la población actual utilizando la antigua población pop y la población recién

generada *newpop*. Tomando prestada la terminología de la comunidad de estrategias evolutivas [194, 202], existen dos posibilidades principales para llevar a cabo esta reconstrucción: la estrategia *plus* y la estrategia *comma*. En la primera, la población actual se construye tomando las mejores *popsizes* configuraciones de  $pop \cup newpop$ . En cuanto a la última, las mejores *popsizes* configuraciones se toman sólo de *newpop*. En este caso, se requiere tener  $|newpop| > popsize$ , a fin de poner cierta presión selectiva en el proceso (cuanto mayor sea la proporción  $|newpop|/popsizes$ , más fuerte será la presión). De lo contrario, la búsqueda se reduciría a un vagabundeo aleatorio a través de  $S$ .

Hay una serie de estudios con respecto a las opciones apropiadas para el procedimiento *UpdatePopulation* (por ejemplo, [9] [78]). Como pauta general, la estrategia *comma* se considera usualmente como menos propensa al estancamiento, siendo la proporción  $|newpop|/popsizes \approx 6$  una opción común [8]. Sin embargo, esta opción puede ser algo computacionalmente costosa si la función guía es compleja y consume mucho tiempo.

Otra alternativa común es usar una estrategia *plus* con un valor bajo de  $|newpop|$ , análoga a la llamada estrategia de reemplazo de estado estacionario en los AGs [222]. Esta opción usualmente proporciona una convergencia más rápida a soluciones de alta calidad. Sin embargo, hay que tener cuidado con la convergencia prematura a regiones subóptimas del espacio de búsqueda, es decir, todas las configuraciones en la población son muy similares entre sí, obstaculizando así la exploración de otras regiones de  $S$ .

La consideración anterior sobre la convergencia prematura conduce al último componente de la plantilla mostrada en la Fig. 2, el procedimiento de reinicio. Antes que nada, debe decidirse si la población se ha degradado

o no. Para ello, es posible usar alguna medida de diversidad de información en la población, como la entropía de Shannon [50]. Si esta medida cae por debajo de un umbral predefinido, se considera que la población se encuentra en un estado degenerado. Este umbral depende de la representación (número de valores por variable, restricciones, etc.) y, por tanto, debe determinarse de forma ad-hoc. Una posibilidad diferente es utilizar un enfoque probabilístico para determinar con una confianza deseada que la población ha convergido. Por ejemplo, en [99] se presenta un enfoque bayesiano con este propósito.

Una vez que se considera que la población se encuentra en un estado de generado, se invoca el procedimiento de reinicio. Una vez más, esto se puede implementar de diversas maneras. Una estrategia muy típica es mantener una fracción de la población actual (esta fracción puede ser tan pequeña como una solución, la mejor actual), y sustituir las configuraciones restantes por soluciones recién generadas

(desde cero), como se muestra en la Fig. 5:

```
Procedure RestartPopulation (pop)
begin
  Initialize newpop using EmptyPopulation();
  #preserved := popsize * %preserve;
  for j := 1 to #preserved do
    i := ExtractBestFromPopulation(pop);
    InsertInPopulation individual i to newpop;
  endfor
  parfor j := #preserved + 1 to popsize do
    i := GenerateRandomConfiguration();
    i := Local-Search-Engine(i);
    InsertInPopulation individual i to newpop;
  endparfor;
  return newpop
end
```

Figura 5: El procedimiento RestartPopulation.

El procedimiento que se muestra en la Fig. 5 también se conoce como la estrategia de *inmigrante-aleatorio* [38]. Una posibilidad diferente es activar un operador de mutación *fuerte* o *pesada* para alejar a la población de su ubicación actual en el espacio de búsqueda. Ambas opciones tienen sus ventajas y desventajas. Por ejemplo, cuando se utiliza la estrategia de inmigrante-aleatorio, hay que tener cierta precaución para evitar que las configuraciones preservadas se apoderen de la población (esto se puede lograr ejerciendo una presión selectiva baja, al menos en las primeras iteraciones después del reinicio). En cuanto a la estrategia de mutación pesada, se debe lograr un equilibrio entre una mutación excesivamente fuerte que destruiría toda la información contenida en la población actual, y una mutación no tan

fuerte que causarí­a que la población convergiera nuevamente en unas pocas iteraciones.

